1. High-Level Design (HLD)

1.1 Project Overview

- The '**PricePulse' Crypto Predictor** is a centralized platform designed to solve the problem of market unpredictability. By leveraging Machine Learning, the system transforms raw historical price data into actionable volatility forecasts.
- The primary goal is to provide a tool that takes raw historical OHLCV (Open, High, Low, Close, Volume) data and outputs a volatility prediction. This helps stakeholders manage risk during periods of market instability.
- **User Interface:** A web dashboard built with **Streamlit**.
- **Data Flow:** The user uploads a CSV →The system calculates technical indicators → The ML model makes a prediction → Results are visualized as a line chart.
- **Technology Stack:** Python, Pandas (Data Manipulation), Scikit-Learn (Modeling), Joblib (Model Storage), Streamlit (Deployment).

1.2 System Architecture

- **Data Layer:** Consists of CSV-based historical data including Open, High, Low, Close, and Volume.
- **Processing Layer:** A Python-based engine that cleans data and calculates complex financial indicators.
- **Model Layer:** A pre-trained Random Forest Regressor that performs pattern recognition on market features.
- **Presentation Layer:** A Streamlit web interface that allows users to interact with the model and visualize results instantly.
- The project follows a **Three-Tier Architecture**:

A. **Data Tier:** Historical CSV data storage and ingestion.
B. **Logic Tier (Application Server):** Python scripts for cleaning, feature engineering, and model inference.
C. **Presentation Tier:** A Streamlit-based frontend for user interaction and data visualization.

1.3 User Flow

1. The user accesses the Streamlit web application.
2. The user uploads a dataset.csv file.
3. The system automatically triggers the feature engineering pipeline.
4. The model generates volatility predictions for every row of data.
5. Interactive charts and summary metrics (Avg/Max Volatility) are displayed.

1.4 Data Flow

- **Ingestion:** The system accepts .csv files via a file uploader component.
- **Validation:** Checking for required columns (Open, High, Low, Volume).
- **Transformation:** Calculating rolling statistics and liquidity metrics.
- **Inference:** Passing the processed data through a serialized Random Forest model.
- **Output:** Rendering time-series charts and key performance metrics.
- **Pipeline Architecture :**

a) **Input:** st.file_uploader receives the CSV.
b) **Transformation:** pandas generates MA_20 and liquidity_ratio.
c) **Loading:** joblib.load('volatility_model.pkl') brings the brain of the app into memory.
d) **Prediction:** model.predict() processes the feature matrix.
e) **Visualization:** st.line_chart renders the time-series volatility graph.

---

2. Low-Level Design (LLD)

2.1 Data Preprocessing Logic

To ensure the model receives high-quality data, the following logic is implemented :

- Handling Nulls: The data.dropna() function is used specifically after feature engineering to remove rows where the 20-day Moving Average (MA_20) cannot be calculated.
- Zero-Division Guard: In the Liquidity Ratio calculation, a constant of (1e-10) is added to the denominator to prevent the application from crashing during periods of zero price movement.
- Preprocessing: Used dropna() to remove rows with missing values and handled division-by-zero errors in the Liquidity Ratio calculation by adding a small constant (1e-10).
- Feature Engineering:
    - MA_20: A 20-day Simple Moving Average to capture price momentum.
    - Liquidity Ratio: Calculated as Volume / (High - Low).
- Model Implementation: Random Forest with 100 decision trees.

To capture the complex nature of crypto markets, the following features were engineered:

- **20-Day Moving Average (MA_20):**

  MA_{20} = \frac{1}{20} \sum_{i=1}^{20} Close_i

  This feature helps the model understand the current price trend relative to the last 20 days.

- **Liquidity Ratio:** Ratio = \frac{Volume}{(High - Low) + 1e^{-10}} This captures market "thinness." A high volume with a small price spread indicates high liquidity and stability.

2.2 Feature Engineering Specifications

- **MA_20 (Technical Indicator):** Calculated using a rolling window of 20 days. This provides the model with a "memory" of recent price trends.
- **Liquidity Ratio:** Defined as Volume / (High - Low). This feature captures the intensity of price movement relative to trading activity.

- **Null Handling:** The application uses data.dropna() specifically after feature engineering to ensure the "warm-up" period of the Moving Average does not feed NaN values into the model.
- **Stability Guard:** A small epsilon value (1e^{-10}) is added to denominators to prevent "Division by Zero" errors during stagnant market periods.

2.3 Model Details

- **Algorithm:** Random Forest Regressor.
- **Parameters:** 100 Decision Trees (n_estimators=100) providing a robust ensemble approach to reduce variance.
- **Deployment:** The model is serialized using the joblib library into a .pkl file for high-speed loading in the production environment.
- **Algorithm:** Random Forest Regressor.
- **Justification:** Chosen for its ability to handle non-linear data and its resistance to overfitting through ensemble learning (bagging).
- **Serialization:** The model is saved using joblib at a compression level that balances loading speed and storage.
- **Future Tuning:** The notebook includes a pre-configured GridSearchCV block to optimize n_estimators and max_depth for future iterations.

3. Performance & Evaluation

The model was evaluated based on its ability to identify volatility "spikes." By utilizing metrics like **Average Volatility** and **Max Volatility** on the dashboard, the user can quickly identify market anomalies.

4. Deployment Strategy

The model is deployed using **Streamlit**, which creates a local web server. This allows for:

- **Interactive Input:** Users can upload their own datasets.
- **Real-time Visualization:** Charts update dynamically based on the uploaded file's content.

Submitted By: Prasad Pradip Sudame

DA MAY-25 Batch